by the model are prioritized by stakeholders and grouped within requirements packages that will be implemented as software increments. A review of the requirements model addresses the following questions:

1. Is each requirement consistent with the overall objectives for the system or product?
2. Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
3. Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
4. Is each requirement bounded and unambiguous?
5. Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
6. Do any requirements conflict with other requirements?
7. Is each requirement achievable in the technical environment that will house the system or product?
8. Is each requirement testable, once implemented?
9. Does the requirements model properly reflect the information, function, and behavior of the system to be built?
10. Has the requirements model been "partitioned" in a way that exposes progressively more detailed information about the system?
11. Have requirements patterns been used to simplify the requirements model? Have all patterns been properly validated? Are all patterns consistent with customer requirements?

These and other questions should be asked and answered to ensure that the requirements model is an accurate reflection of stakeholder needs and that it provides a solid foundation for design.

7.9 SUMMARY

Requirements engineering tasks are conducted to establish a solid foundation for design and construction. Requirements engineering occurs during the communication and modeling activities that have been defined for the generic software process. Seven requirements engineering activities—inception, elicitation, elaboration, negotiation, specification, validation, and management—are conducted by members of the software team and product stakeholders.

At project inception, stakeholders establish basic problem requirements, define overriding project constraints, and address major features and functions that must be present for the system to meet its objectives. This information is refined and expanded during elicitation—a requirements gathering activity that makes use of facilitated meetings and the development of usage scenarios (user stories).

Elaboration further expands requirements in a model—a collection of scenario-based, activity-based, class-based, and behavioral elements. The model may reference

analysis patterns, characteristics of the problem domain that have been seen to reoccur across different applications.

As requirements are identified and the requirements model is being created, the software team and other project stakeholders negotiate the priority, availability, and relative cost of each requirement. The intent of this negotiation is to develop a realistic project plan. Each requirement needs to be validated against customer needs to ensure that the right system is to be built.

PROBLEMS AND POINTS TO PONDER

7.1. Why is it that many software developers don't pay enough attention to requirements engineering? Are there ever circumstances where you can skip it?

7.2. You have been given the responsibility to elicit requirements from a customer who tells you he is too busy to meet with you. What should you do?

7.3. Discuss some of the problems that occur when requirements must be elicited from three or four different customers.

7.4. Your instructor will divide the class into groups of four or six students. Half of the group will play the role of the marketing department and half will take on the role of software engineering. Your job is to define requirements for the *SafeHome* security function described in this chapter. Conduct a requirements gathering meeting using the guidelines presented in this chapter.

7.5. Develop a complete use case for one of the following activities:

a. Making a withdrawal at an ATM
b. Using your charge card for a meal at a restaurant
c. Searching for books (on a specific topic) using an online bookstore

7.6. Write a user story for one of the activities listed in Problem 7.5.

7.7. Consider the use case you created in Problem 7.5, and write a nonfunctional requirement for the application.

7.8. Using the template presented in Section 7.5.2, suggest one or more analysis patterns for the following application domains:

a. E-mail software.
b. Internet browsers.
c. Mobile app creation software.

7.9. What does *win-win* mean in the context of negotiation during the requirements engineering activity?

7.10. What do you think happens when requirement validation uncovers an error? Who is involved in correcting the error?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

8 REQUIREMENTS MODELING—A RECOMMENDED APPROACH

The written word is a wonderful vehicle for communication, but it is not necessarily the best way to represent the requirements for computer software. At a technical level, software engineering begins with a series of modeling tasks that lead to a specification of requirements and a design representation for the software to be built. The requirements model is actually a set of models that make up the first technical representation of a system. Software engineers often prefer to include graphical representations of complex model relationships.

KEY CONCEPTS

activity diagram 146
analysis classes 137
attributes 140
behavioral model 149
class-based modeling 127
collaborations 144
CRC modeling 144
events 149
formal use case 135
functional model 146
grammatical parse 137
operations 140
procedural view 146
requirements analysis 127
requirements modeling 129
responsibilities 144
scenario-based modeling 128
sequence diagrams 148
state diagrams 150
swimlane diagram 151
UML models 130
use cases 131
  documentation 135
  exception 134



QUICK LOOK

What is it? Requirements modeling uses a combination of text and diagrammatic forms to depict requirements in a way that is relatively easy to understand, and more important, straightforward to review for correctness, completeness, and consistency.

Who does it? A software engineer (sometimes called an analyst) builds these models from requirements elicited from various stakeholders.

Why is it important? Requirements models can be readily evaluated by all stakeholders, resulting in useful feedback at the earliest possible time. Later, as the model is refined, it becomes the basis for software design.

What are the steps? Requirements modeling combines three steps: scenario-based modeling, class modeling, and behavioral modeling.

What is the work product? Usage scenarios, called use cases, describe software functions and usage. In addition, a series of UML diagrams can be used to represent system behavior and other aspects.

How do I ensure that I've done it right? Requirements modeling work products must be reviewed for correctness, completeness, and consistency.

For some types of software, a user story (Section 7.3.2) may be the only requirements modeling representation that is required. For others, formal use cases (Section 7.4) and class-based models (Section 8.3) may be developed. Class-based modeling represents the objects that the system will manipulate, the operations (also called *methods* or *services*) that will be applied to the objects to effect the manipulation, relationships (some hierarchical) between the objects, and the collaborations that occur between the classes that are defined. Class-based methods can be used to create a representation of an application that can be understood by nontechnical stakeholders.

In still other situations, complex application requirements may demand an examination of how an application behaves in reaction to either internal or external events. These behaviors need to be modeled (Section 8.5) as well. UML diagrams have become a standard software engineering means of modeling analysis model element relationships and behaviors graphically. As the requirements model is refined and expanding, it evolves into a specification that can be used by software engineers in the creation of the software design.

The important thing to keep in mind when modeling requirements is to only create the models that will be used by the development team. If models developed early in a requirements analysis phase of a project are not referred to during the design and implementation phases, they may not be worth updating. The sections that follow present a series of informal guidelines that will assist in the creation and representation of requirements models.

8.1 REQUIREMENTS ANALYSIS

Requirements analysis results in the specification of software's operational characteristics, indicates software's interface with other system elements, and establishes constraints that software must meet. Requirements analysis allows you (regardless of whether you're called a *software engineer,* an *analyst,* or a *modeler*) to elaborate on basic requirements established during the inception, elicitation, and negotiation tasks that are part of requirements engineering (Chapter 7).

The requirements modeling action results in one or more of the following types of models:

- *Scenario-based models* of requirements from the point of view of various system "actors."
- *Class-oriented models* that represent object-oriented classes (attributes and operations) and how classes collaborate to achieve system requirements.
- *Behavioral models* that depict how the software reacts to internal or external "events."
- *Data models* that depict the information domain for the problem.
- *Flow-oriented models* that represent the functional elements of the system and how they transform data as they move through the system.

These models provide a software designer with information that can be translated to architectural-, interface-, and component-level designs. Finally, the requirements

model (and the software requirements specification) provides the developer and the customer with the means to assess quality once software is built.

In this section, we focus on *scenario-based modeling*—a technique that is very popular throughout the software engineering community. In Sections 8.3 and 8.5 we consider class-based modeling and behavioral modeling. Over the past decade, flow and data modeling have become less commonly used, while scenario and class-based methods, supplemented with behavioral approaches have grown in popularity.[1]

8.1.1 Overall Objectives and Philosophy

Throughout analysis modeling, your primary focus is on *what,* not *how.* What user interaction occurs, what objects does the system manipulate, what functions must the system perform, what behaviors does the system exhibit, what interfaces are defined, and what constraints apply?[2]

In previous chapters, we noted that complete specification of requirements may not be possible at this stage. The customer may be unsure of precisely what is required for certain aspects of the system. The developer may be unsure that a specific approach will properly accomplish function and performance. These realities mitigate in favor of an iterative approach to requirements analysis and modeling. The analyst should model what is known and use that model as the basis for design of the software increment.[3]

The requirements model must achieve three primary objectives: (1) to describe what the customer requires, (2) to establish a basis for the creation of a software design, and (3) to define a set of requirements that can be validated once the software is built. The analysis model bridges the gap between a system-level description that describes overall system or business functionality (software, hardware, data, human elements) and a software design (Chapters 9 through 14) that describes the software's application architecture, user interface, and component-level structure. This relationship is illustrated in Figure 8.1.

It is important to note that all elements of the requirements model will be directly traceable to parts of the design model. A clear division between analysis and design modeling tasks is not always possible. Some design invariably occurs as part of analysis, and some analysis will be conducted during design.

8.1.2 Analysis Rules of Thumb

Several rules of thumb [Arl02] are worth considering when creating an analysis model. First, focus on the problem or business domain while keeping the level of abstraction high. Second, recognize that an analysis model should provide insight into

---

1 A presentation of flow-oriented modeling and data modeling is no longer included in this chapter. However, copious information about these older requirements modeling methods can be found on the Web. If you have interest, use the search phrase "structured analysis."

2 It should be noted that as customers become more technologically sophisticated, there is a trend toward the specification of *how* as well as *what.* However, the primary focus should remain on *what.*

3 Alternatively, the software team may choose to create a prototype (Chapter 4) to better understand requirements for the system.

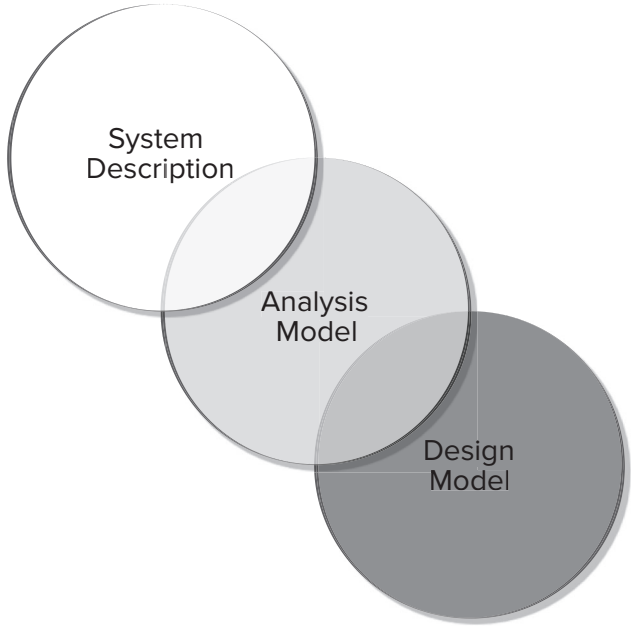


FIGURE 8.1
The analysis model as a bridge between the system description and the design model

the information domain, the function, and the behavior of the software. Third, delay a consideration of software architecture and nonfunctional details until later in the modeling activity. Also, it's important to be aware of the ways in which elements of the software are interconnnected with other elements (we call this *system coupling*).

The analysis model must be structured in a way that provides value to all stakeholders and should be kept as simple as possible without sacrificing clarity.

8.1.3 Requirements Modeling Principles

Over the past four decades, several requirements modeling methods have been developed. Investigators have identified requirements analysis problems and their causes and have developed a variety of modeling notations and corresponding sets of heuristics to overcome them. Each analysis method has a unique point of view. A set of operational principles relates analysis methods:

Principle 1. ***The information domain of a problem must be represented and understood.*** The *information domain* encompasses the data that flow into the system (from end users, other systems, or external devices), the data that flow out of the system (via the user interface, network interfaces, reports, graphics, and other means), and the data stores that collect and organize the data that are maintained permanently.

Principle 2. ***The functions that the software performs must be defined.*** Software functions provide direct benefit to end users and those that provide internal support for those features that are user visible. Some functions transform data that